



BFS, Shortest Paths & Spanning Trees

Lecture 5 | Algorithms Design & Analysis

Girish Varma
IIIT Hyderabad

Figure 4.3 Breadth-first search.

procedure `bfs`(G, s)

Input: Graph $G = (V, E)$, directed or undirected; vertex $s \in V$

Output: For all vertices u reachable from s , `dist`(u) is set
 to the distance from s to u .

for all $u \in V$:

`dist`(u) = ∞

`dist`(s) = 0

$Q = [s]$ (queue containing just s)

while Q is not empty:

$u = \text{eject}(Q)$

 for all edges $(u, v) \in E$:

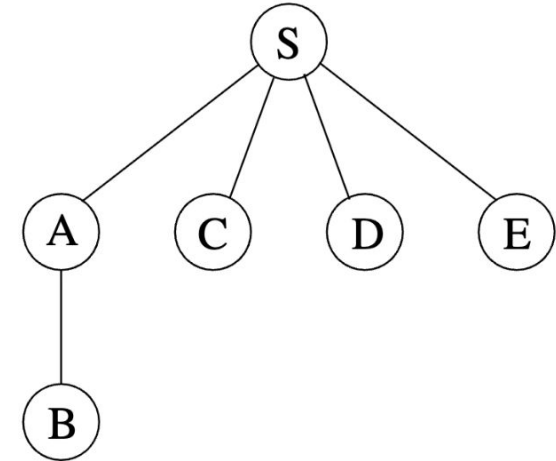
 if `dist`(v) = ∞ :

`inject`(Q, v)

`dist`(v) = `dist`(u) + 1

Figure 4.4 The result of breadth-first search on the graph of Figure 4.1.

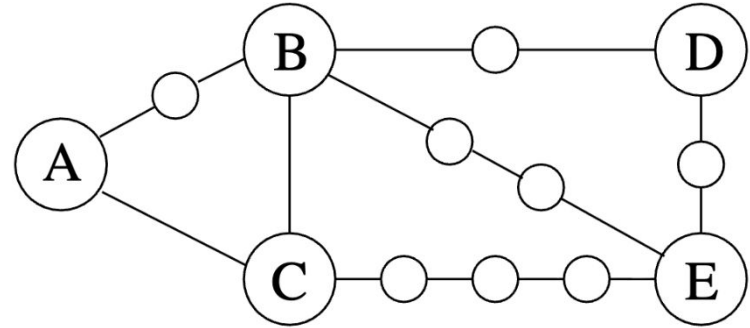
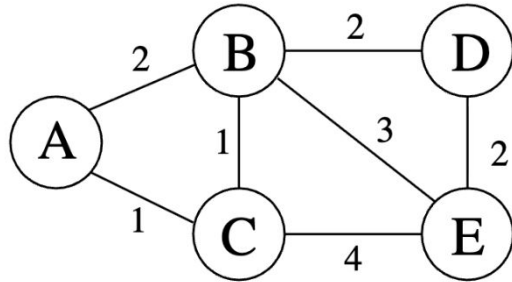
Order of visitation	Queue contents after processing node
<i>S</i>	[<i>S</i>]
<i>A</i>	[<i>A C D E</i>]
<i>C</i>	[<i>C D E B</i>]
<i>D</i>	[<i>D E B</i>]
<i>E</i>	[<i>E B</i>]
<i>B</i>	[<i>B</i>]
	[]



Shortest Paths



Figure 4.6 Breaking edges into unit-length pieces.



Dijkstra's Algorithm



Figure 4.8 Dijkstra's shortest-path algorithm.

procedure `dijkstra`(G, l, s)

Input: Graph $G = (V, E)$, directed or undirected;
positive edge lengths $\{l_e : e \in E\}$; vertex $s \in V$

Output: For all vertices u reachable from s , $\text{dist}(u)$ is set
to the distance from s to u .

for all $u \in V$:

$\text{dist}(u) = \infty$

$\text{prev}(u) = \text{nil}$

$\text{dist}(s) = 0$

$H = \text{makequeue}(V)$ (using dist -values as keys)

while H is not empty:

$u = \text{deletemin}(H)$

 for all edges $(u, v) \in E$:

 if $\text{dist}(v) > \text{dist}(u) + l(u, v)$:

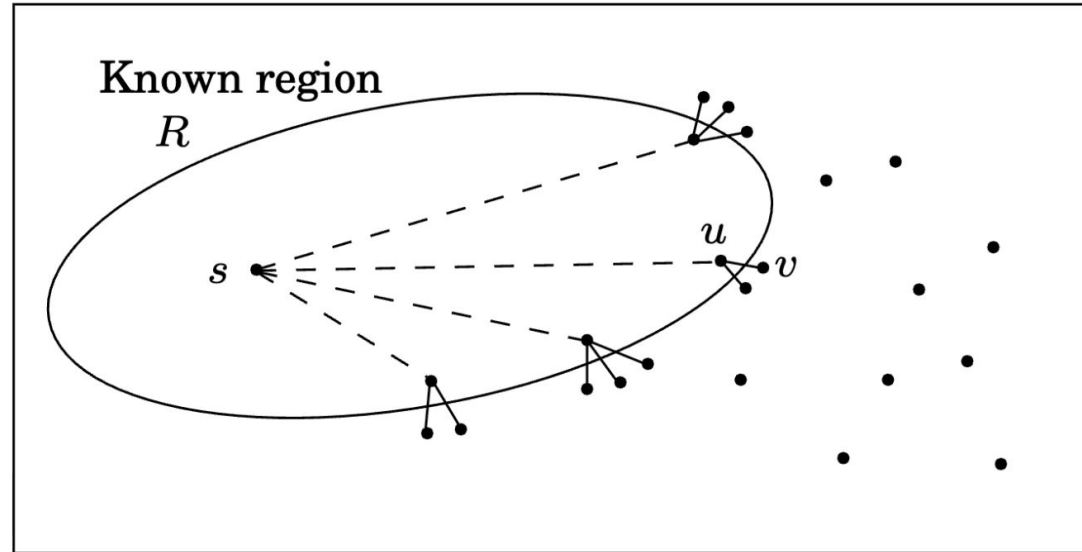
$\text{dist}(v) = \text{dist}(u) + l(u, v)$

$\text{prev}(v) = u$

$\text{decreasekey}(H, v)$

<https://dijkstra-sim.vercel.app/>

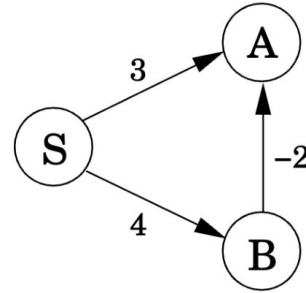
Figure 4.10 Single-edge extensions of known shortest paths.



Negative Edges



Figure 4.12 Dijkstra's algorithm will not work if there are negative edges.



Dijkstra's algorithm works in part because the shortest path from the starting point s to any node v must pass exclusively through nodes that are closer than v . This no longer holds when edge lengths can be negative. In [Figure 4.12](#), the shortest path from S to A passes through B , a node that is further away!

Update Operation



procedure update $((u, v) \in E)$
 $\text{dist}(v) = \min\{\text{dist}(v), \text{dist}(u) + l(u, v)\}$

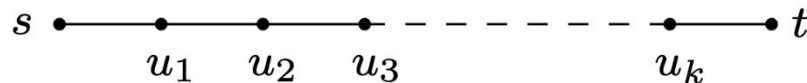
This *update* operation is simply an expression of the fact that the distance to v cannot possibly be more than the distance to u , plus $l(u, v)$. It has the following properties.

1. It gives the correct distance to v in the particular case where u is the second-last node in the shortest path to v , and $\text{dist}(u)$ is correctly set.
2. It will never make $\text{dist}(v)$ too small, and in this sense it is *safe*. For instance, a slew of extraneous update's can't hurt.

Update Operation



node t and look at the shortest path to it from s .



This path can have at most $|V| - 1$ edges (do you see why?). If the sequence of updates performed includes $(s, u_1), (u_1, u_2), (u_2, u_3), \dots, (u_k, t)$, *in that order* (though not necessarily consecutively), then by the first property the distance to t will be correctly computed. It doesn't

But still, if we don't know all the shortest paths beforehand, how can we be sure to update the right edges in the right order? Here is an easy solution: simply update *all* the edges, $|V| - 1$ times! The resulting $O(|V| \cdot |E|)$ procedure is called the Bellman-Ford algorithm and is shown in [Figure 4.13](#), with an example run in [Figure 4.14](#).

Bellman Ford Single Source Shortest Paths



Figure 4.13 The Bellman-Ford algorithm for single-source shortest paths in general graphs.

procedure shortest-paths(G, l, s)

Input: Directed graph $G = (V, E)$;

edge lengths $\{l_e : e \in E\}$ with no negative cycles;

vertex $s \in V$

Output: For all vertices u reachable from s , $\text{dist}(u)$ is set
to the distance from s to u .

for all $u \in V$:

$\text{dist}(u) = \infty$

$\text{prev}(u) = \text{nil}$

$\text{dist}(s) = 0$

repeat $|V| - 1$ times:

 for all $e \in E$:

$\text{update}(e)$

Negative Cycles



Fortunately, it is easy to automatically detect negative cycles and issue a warning. Such a cycle would allow us to endlessly apply rounds of update operations, reducing `dist` estimates every time. So instead of stopping after $|V| - 1$ iterations, perform one extra round. There is a negative cycle if and only if some `dist` value is reduced during this final round.

Shortest Paths in DAGs



In any path of a dag, the vertices appear in increasing linearized order.

Figure 4.15 A single-source shortest-path algorithm for directed acyclic graphs.

procedure dag-shortest-paths(G, l, s)

Input: Dag $G = (V, E)$;

 edge lengths $\{l_e : e \in E\}$; vertex $s \in V$

Output: For all vertices u reachable from s , $\text{dist}(u)$ is set
 to the distance from s to u .

for all $u \in V$:

$\text{dist}(u) = \infty$

$\text{prev}(u) = \text{nil}$

$\text{dist}(s) = 0$

Linearize G

for each $u \in V$, in linearized order:

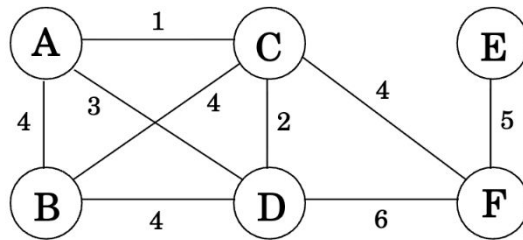
 for all edges $(u, v) \in E$:

$\text{update}(u, v)$

Minimum Spanning Tree



Suppose you are asked to network a collection of computers by linking selected pairs of them. This translates into a graph problem in which nodes are computers, undirected edges are potential links, and the goal is to pick enough of these edges that the nodes are connected. But this is not all; each link also has a maintenance cost, reflected in that edge's weight. What is the cheapest possible network?



One immediate observation is that the optimal set of edges cannot contain a cycle, because removing an edge from this cycle would reduce the cost without compromising connectivity:

Kruskal's Algorithm



Repeatedly add the next lightest edge that doesn't produce a cycle.

Cut Property



Cut property Suppose edges X are part of a minimum spanning tree of $G = (V, E)$. Pick any subset of nodes S for which X does not cross between S and $V - S$, and let e be the lightest edge across this partition. Then $X \cup \{e\}$ is part of some MST.

A *cut* is any partition of the vertices into two groups, S and $V - S$. What this property says is that it is always safe to add the lightest edge across any cut (that is, between a vertex in S and one in $V - S$), provided X has no edges across the cut.

Kruskal's Algorithm



Figure 5.4 Kruskal's minimum spanning tree algorithm.

procedure `kruskal`(G, w)

Input: A connected undirected graph $G = (V, E)$ with edge weights w_e

Output: A minimum spanning tree defined by the edges X

for all $u \in V$:

`makeset`(u)

$X = \{\}$

Sort the edges E by weight

for all edges $\{u, v\} \in E$, in increasing order of weight:

 if `find`(u) $\neq$ `find`(v):

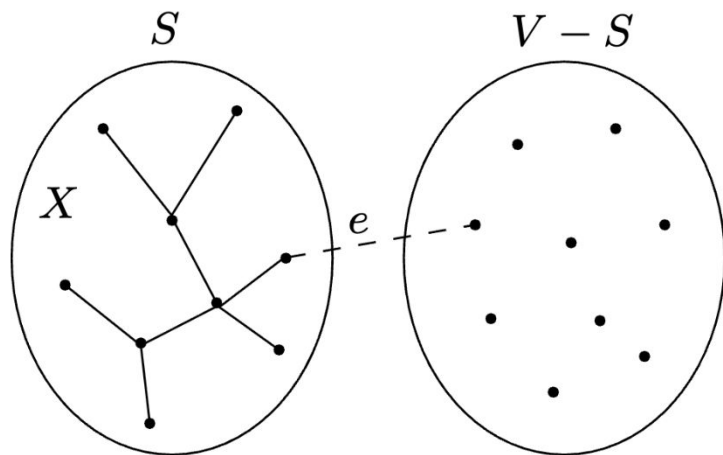
 add edge $\{u, v\}$ to X

`union`(u, v)

Prim's Algorithm



Figure 5.8 Prim's algorithm: the edges X form a tree, and S consists of its vertices.



Prim's Algorithm



Figure 5.9 *Top:* Prim's minimum spanning tree algorithm. *Below:* An illustration of Prim's algorithm, starting at node A . Also shown are a table of cost/prev values, and the final MST.

procedure prim(G, w)

Input: A connected undirected graph $G = (V, E)$ with edge weights w_e

Output: A minimum spanning tree defined by the array `prev`

for all $u \in V$:

`cost`(u) = ∞

`prev`(u) = nil

Pick any initial node u_0

`cost`(u_0) = 0

$H = \text{makequeue}(V)$ (priority queue, using cost-values as keys)

while H is not empty:

$v = \text{deletemin}(H)$

 for each $\{v, z\} \in E$:

 if `cost`(z) > $w(v, z)$:

`cost`(z) = $w(v, z)$

`prev`(z) = v

`decreasekey`(H, z)

Read



Chapter 4, Chapter 5 Section 5.1 in

<http://algorithmics.isi.upc.edu/docs/Dasgupta-Papadimitriou-Vazirani.pdf>

Solve



Exercises 4.5, 4.10, 4.13, 4.17, 4.21, 4.22

Exercises 5.5, 5.6, 5.9 in

<http://algorithmics.isi.upc.edu/docs/Dasgupta-Papadimitriou-Vazirani.pdf>